

Git basics

STATS 21

Fall 2026

Miles Chen

What is Git

- Git is a version control system.
- A commit records a snapshot of the files you staged.
- History lets you inspect earlier versions and changes.
- Git supports collaboration through shared repositories.
- Saving locally changes a file. Committing to a git repository records local history.
- Pushing uploads commits to your GitHub repository.

The very basics

```
git status
```

Check before and after staging, committing, or pulling.

```
git add name_of_edited_file
```

Stage the saved changes in the file you intend to commit.

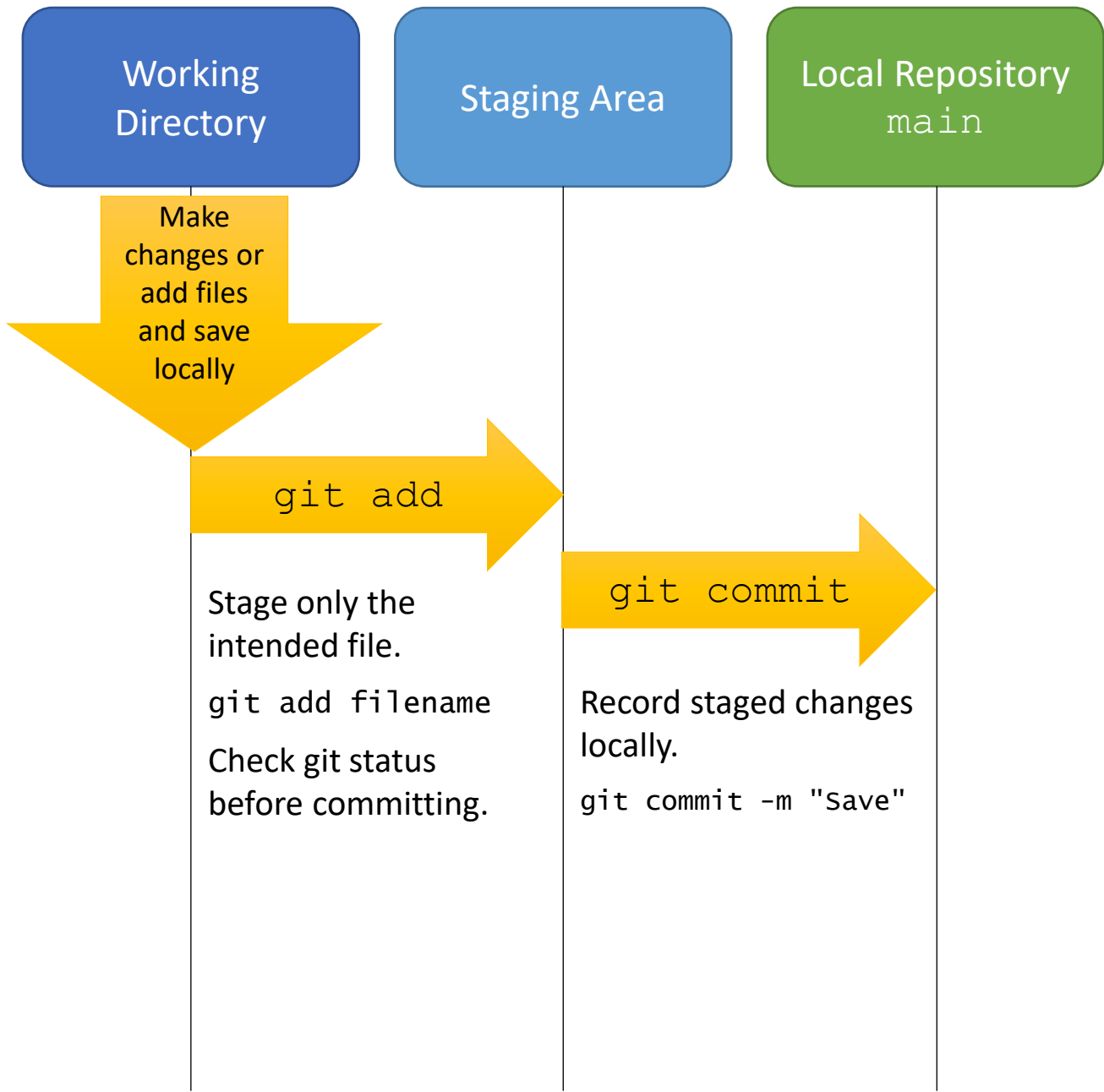
```
git status
```

Check that only intended files are staged

```
git commit -m "write a comment describing the edit"
```

Record the staged changes in your local history.

Saving changes locally



Why add before commit

- A commit should represent a small, logical change.
- The staging area lets you choose what belongs together. You may be working on several files at once, but they do not all need to be committed together.
- Save your work to your local machine, then use `'git add'` to add only the intended files.
- Always use a message that explains the change in `'git commit'`.

`git log --oneline`

- Git log allows you to read the current branch's commit history.
- If a log opens in a pager, press q to exit.

Recovery from an accidental edit

First inspect git status and preserve any useful content.

Only for an original whose edits are unstaged, uncommitted, and unneeded.

```
git restore -- file_name
```

```
git status
```

This discards one file's unstaged edits

Do not use 'git restore .' as a general fix.

For conflicts, staged originals, or unclear status: stop and ask for help. AI tools like ChatGPT are quite helpful at resolving issues with git.

Reading changes in a commit

```
git show 123abc
```

Replace 123abc with a commit ID from git log.

Git shows the author, date, message, and changes.

```
--- a/filename.txt
```

The file before the change.

```
+++ b/filename.txt
```

The file after the change.

```
- A removed line
```

```
+ An added line
```

A leading space marks unchanged context.

A diff hunk header such as @@ -23,4 +23,5 @@ gives the old starting line and count, followed by the new starting line and count

Using GitHub

GitHub, forks, and clones

- GitHub hosts remote repositories that use Git.
- A fork is your copy of a repository on GitHub.
- A clone is a local copy on your computer.
- For this class: fork the Fall 2026 class repository, then clone your fork.
- origin: your GitHub fork, where you save your work.
- upstream: the professor's repository, where you get updates.

Getting set up

- Follow 00_setting_up.pdf for the complete setup.
- Use Terminal on macOS or PowerShell on Windows.
- Create a GitHub account, then install and configure Git.

```
git --version
```

```
git config --global user.name "Your Name"
```

```
git config --global user.email "you@example.com"
```

- Use your own name and an email linked to GitHub.
- The guide also sets up VS Code, uv, and the class .venv.

Creating an SSH key

Check for an existing key before generating a new one.

macOS

```
ls -al ~/.ssh
```

Windows PowerShell

```
Get-ChildItem -Force "$HOME\.ssh"
```

Reuse id_ed25519 and id_ed25519.pub if appropriate; never overwrite an existing key.

If no key exists, use your own email and run:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Best practice recommends a passphrase. For this course, I recommend leaving it blank for convenience.

Keep your computer secure. Never share the private key.

Public keys and the SSH connection test

Copy the public key (.pub), never the private key.

macOS:

```
pbcopy < ~/.ssh/id_ed25519.pub
```

Windows PowerShell:

```
Get-Content "$HOME\.ssh\id_ed25519.pub" | Set-Clipboard
```

GitHub: Settings > SSH and GPG keys > New SSH key. Use a computer-specific title and Authentication Key.

```
ssh -T git@github.com
```

[Check GitHub's published host fingerprint before accepting. Then type 'yes'](#)

Success names your account; no shell access is normal.

Forking and cloning the class repository

Before cloning

Sign in to GitHub and complete the SSH test.

Choose a parent folder outside any existing class clone.

Step 1. Fork the class repository on GitHub.

[smileschen/26-fall-stats-21](https://github.com/smileschen/26-fall-stats-21)

Your Remote
Repository on GitHub
origin/main

`git clone`

Step 2. Replace your_github_name, then run:

```
git clone git@github.com:your_github_name/26-fall-stats-21.git  
cd 26-fall-stats-21
```

Your computer: new 26-fall-stats-21 folder

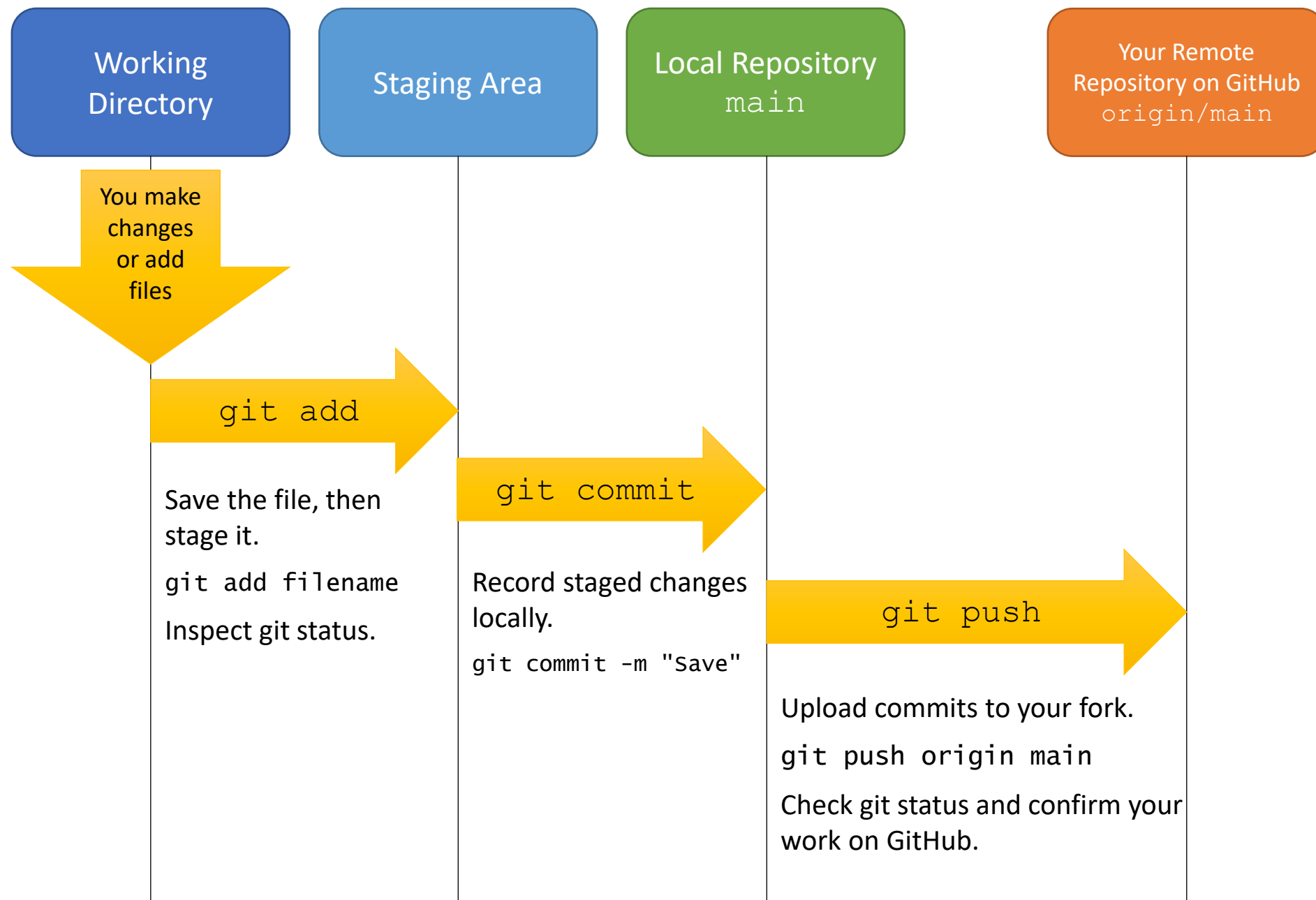
Run later commands from the root of this folder.

Working
Directory

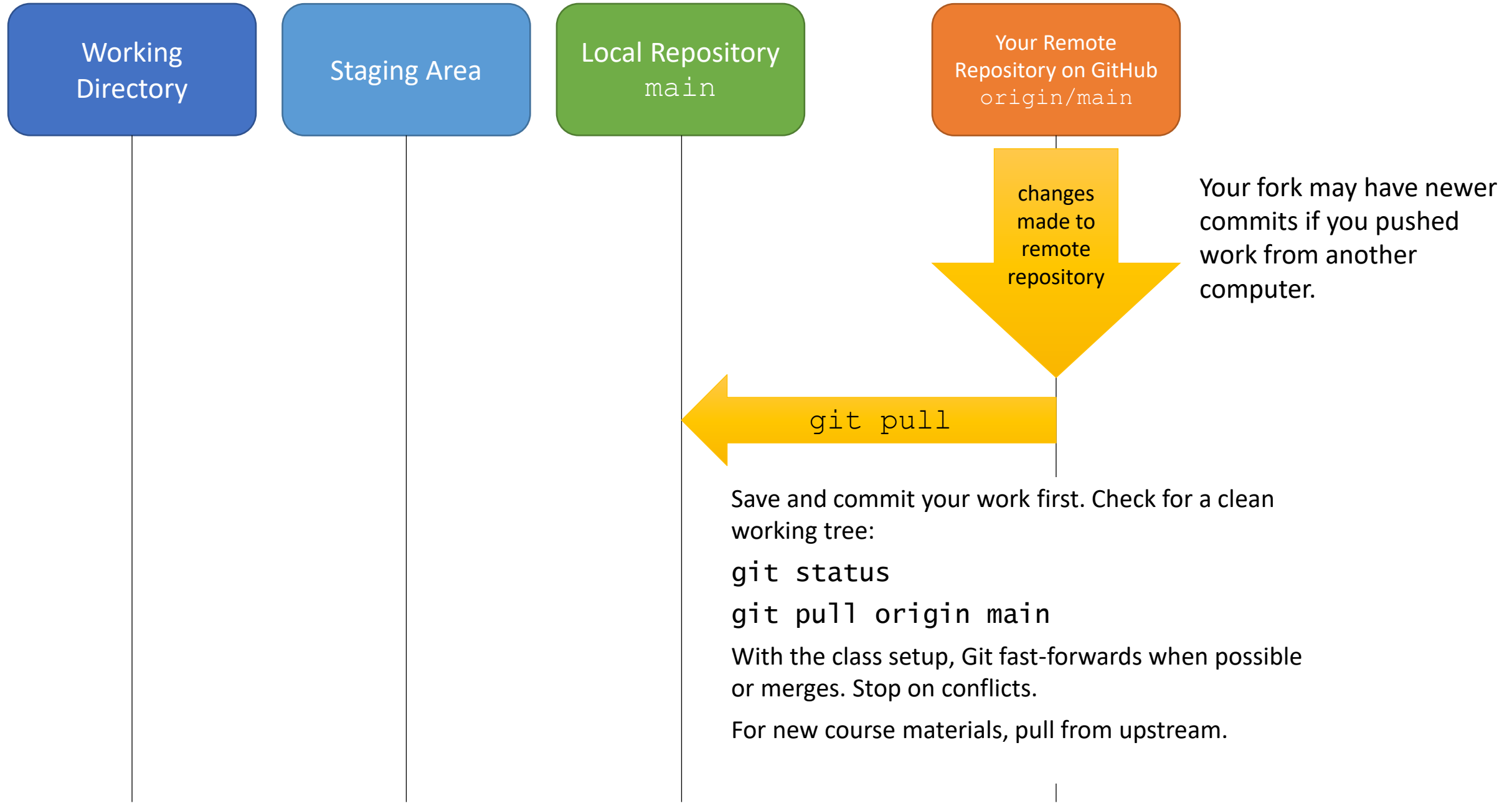
Staging Area

Local Repository
main

Saving work to your GitHub fork



Pulling changes from your GitHub fork



One-time class repository setup

From the root of your cloned 26-fall-stats-21 folder:

```
git remote add upstream git@github.com:smileschen/26-fall-stats-21.git
```

```
git config --local branch.main.rebase false
```

```
git config --local pull.ff true
```

```
git config --local branch.main.mergeOptions "--no-edit"
```

Pulls fast-forward when possible; otherwise they merge.

```
git rev-parse --show-toplevel
```

```
git branch --show-current
```

```
git remote -v
```

Check the class folder, main, and both remote accounts.

The daily class workflow

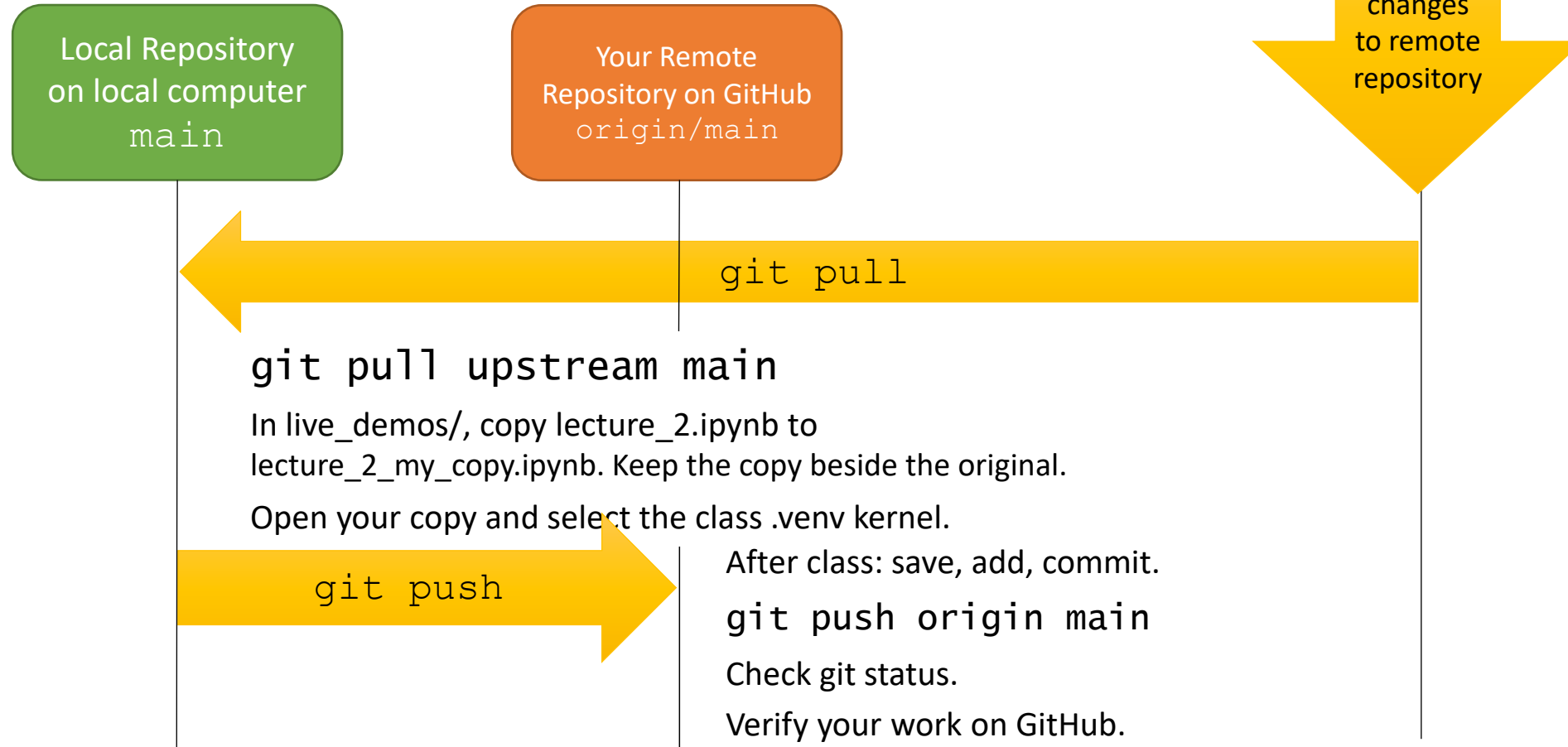
At the start of class

Save open notebooks and run git status.

Commit changes to your own copies before pulling.

Continue when the working tree is clean.

Stop for conflicts or changes to original files.



Your work on GitHub

- After pushing, open your GitHub fork and refresh.
- Confirm that your saved notebook appears there.
- Open the commit history and select a commit to inspect its changes.
- A successful SSH test alone does not upload your work.
- A push is not automatically a homework submission.
- Files in a public fork are public. Upload homework solutions only when the assignment permits it.